

Instructions D'installation et de fonctionnement de l'Arduino Météo

ARRUZAS Simon

Table des matières

I.	Objectifs :	3
II.	Installation :	4
III.	Programmes :	5
IV.	Résultats :	23
V.	Conclusion :	25

I. Objectifs :

L'objectif de ce projet est de réaliser un montage à partir d'un Arduino UNO R4 Minima capable de récupérer des données provenant d'une API d'une station météo CR1000 Datalogger et de divers capteurs afin de les communiquer en LoRa sous forme d'une payload hexadécimale. Les données vont par la suite être décodées grâce à un codec présent sur Chirpstack puis exploitées selon les besoins.

II. Installation :

La configuration est répartie en trois étages :

- **L'Arduino Uno** à la base, cerveau et source d'énergie pour la configuration, c'est avec lui que nous communiquons afin d'exécuter des programmes nommés « sketches ».
- **L'Ethernet Shield** placé juste au-dessus. Il permet à la configuration d'avoir un accès Ethernet.
- **Le Base Shield** situé au sommet, c'est un hub accueillant tous les capteurs analogiques.

On retrouve branché au Base Shield un bouton ainsi que plusieurs capteurs et une antenne :

- **Le Pyranomètre**, capable de mesurer le taux de radiations
- **Le MS5637**, capable de mesurer la pression.
- **Le SEN0636**, capable de mesurer l'indice UV.
- **Le pluviomètre** mesurant la quantité de pluie.
- **Le Wio-E5** permettant la connexion en LoRa.

Une fois le montage terminé, vous pouvez connecter l'**Arduino Uno** à un ordinateur afin de pouvoir accéder à l'IDE.

Depuis cet IDE, vous pouvez faire des mises à jour et vous pouvez maintenant importer toutes les libraries présentes dans un dossier crée pour l'occasion et disponible dans ce repository.

Faites bien attention à ajouter toutes les libraries sans oublier les libraries disponibles uniquement dans l'IDE Arduino, sans cela les programmes risquent de ne pas pouvoir fonctionner.

III. Programmes :

1. Programmes de Tests :

Une fois les libraries installées, vous aurez accès à des programmes de tests permettant d'essayer les composants pour vérifier qu'ils fonctionnent.

En plus de ces programmes de tests, ce repo possède un dossier comprenant d'autres sketch fait sur-mesure pour la configuration.

2. Programmes Principaux :

Il y a trois programmes principaux :

1. **Reader_TrueAPI** : Programme principal recevant des données depuis l'API du Datalogger et depuis les capteurs, il encode les données sous forme d'une chaîne de caractères hexadécimaux avant de les envoyer en LoRa. Il dispose d'un bouton d'arrêt d'urgence qui envoie les données sans aucun délai. Sans interruption, le cycle envoie des données toutes les 13 minutes. On peut observer ce qu'envoie l'API directement dans le Serial Monitor.
2. **Reader_Urgent_Flag_PI_UART** : Programme fonctionnant exactement comme le « **Reader_TrueAPI** » mais fonctionnant de pair avec une fausse API. C'est le programme de test, il est sujet aux mises à jour et aux premières modifications avant de les appliquer sur le « TrueAPI ».
3. **TestAPI** : Programme de test de l'API, il va afficher dans le serial monitor tout ce que capte l'Arduino provenant de l'API. Grâce à celui-ci, nous allons pouvoir avoir un aperçu de ce à quoi ressemble la payload afin de l'exploiter au mieux.

3. Fonctionnement des programmes :

Les deux premiers programmes principaux fonctionnent globalement de la même manière :

On commence par définir les pins et inclure les libraries exploitées :

```
5
6  ✓ #include <SPI.h>
7    #include <Arduino.h>
8    #include <Ethernet.h>
9    #include <BaroSensor.h>
10   #include "DFRobot_UVIndex240370Sensor.h"
11   #include <Wire.h>
12   #include <SoftwareSerial.h>
13
14
15   #define ENABLE_DIAGNOSTICS 1
16   #define VERBOSE_MODE 1
17   #define PYRANO_PIN A0
18   // #define PYRANO_PIN_2 A1
19   #define PYRANO_CALIBRATION 0.4
20   #define RAIN_GAUGE_PIN 8
21   #define RAIN_PER_PULSE 0.2
22   #define DEBOUNCE_TIME 50
23
24   // ===== BUTTON CONFIG =====
25   #define BUTTON_PIN 7
26   #define BUTTON_COOLDOWN 10000 // 10 secondes entre 2 appuis
27
28   // ===== Codes d'alerte =====
29   #define ALERT_BATTERY_LOW 0xFF01 // Code d'alerte batterie faible
```

On stocke ensuite les commandes LoRa en mémoire Flash (Progmem) au lieu de RAM pour gagner un peu d'espace puis on définit les buffers, les flags et les variables d'interruption :

```
// ===== Commandes Progmem permettant de gagner de la RAM =====
const char PROGMEM STR_AT[] = "AT\r\n";
const char PROGMEM STR_ID[] = "AT+ID\r\n";
const char PROGMEM STR_MODE[] = "AT+MODE=LWOTAA\r\n";
const char PROGMEM STR_DR[] = "AT+DR=EUB68\r\n";
const char PROGMEM STR_KEY[] = "AT+KEY=APPKEY,\"96395F2F6686099B84766352E937626E\"\\r\\n";
const char PROGMEM STR_CLASS[] = "AT+CLASS=A\\r\\n";
const char PROGMEM STR_PORT[] = "AT+PORT=8\\r\\n";
const char PROGMEM STR_JOIN[] = "AT+JOIN\\r\\n";

// ===== BUFFERS =====
static char recv_buf[200];
char rxBuffer[1024];
int rxIndex = 0;

// ===== STATE FLAGS =====
static bool lora_module_exists = false;
static bool lora_network_joined = false;
static bool BattV_Avg_is_low = false;

// ===== INTERRUPT VARIABLES =====
volatile bool buttonPressed = false;
unsigned long lastInterruptTime = 0;
unsigned long lastBatteryAlertTime = 0;
```

On définit l'adresse MAC de l'Arduino, l'adresse de l'hôte correspondant à la station météo, son port et on initialise les données des capteurs et le temps total d'un cycle (ici 13 minutes pour s'adapter au Datalogger qui possède un long cycle d'après les ressources mises à ma disposition) :

```

56 // ===== NETWORK CONFIG =====
57 byte mac[] = { 0xA8, 0x61, 0x0A, 0xAE, 0xD9, 0xAD };
58 const char* TCP_HOST = "150.100.100.20"; // Avec le PI : "192.168.1.1"; // Vraie API : 150.100.100.20
59 const int TCP_PORT = 80; // Avec le Pi : 8765; // Mettre 80 pour l'API
60 EthernetClient ethClient;
61
62 // ===== Données des capteurs =====
63 float ms5637_pres = 0; // MS5637 Data
64 uint16_t uv_voltage = 0;
65 uint16_t uv_index = 0;
66 int rawValue = 0;
67 float voltage_mV = 0, radiation_Wm2 = 0;
68 // float voltage_mV_2 = 0, radiation_Wm2_2 = 0; // Pour un second pyranomètre
69 float api_vals[9] = {0}; // 9 values from API
70
71 // Variables globales pluviomètre
72 volatile int rain_pulses = 0;
73 float rain_mm = 0;
74 unsigned long last_rain_time = 0;
75
76 // ===== HARDWARE OBJECTS =====
77 SoftwareSerial loraSerial(2, 3);
78 DFRobot_UVIndex240370Sensor UVIndex240370Sensor(&Serial1);
79
80 // ===== TIMING =====
81 unsigned long lastSendTime = 0;
82 const unsigned long SEND_INTERVAL = 780000; // Pour 13 minutes, modifiable
83

```

Ces fonctions permettent de rendre l'affichage plus lisible dans la console, la fonction rainGaugeISR(Interrupt Service Routine) compte les pulsations du pluviomètre avec un anti-rebond de 50ms pour éviter les faux comptages :

```

84
85 // Permet d'afficher des séparateurs dans la console pour une meilleure lisibilité
86 void printSeparator() {
87     Serial.println("=====");
88 }
89
90 // Permet d'afficher des titres de sections dans la console pour une meilleure lisibilité
91 void printHeader(const char* title) {
92     printSeparator();
93     Serial.print("| ");
94     Serial.print(title);
95     Serial.println(" |");
96     printSeparator();
97 }
98
99 // Permet d'afficher des messages de diagnostic avec un format uniforme
100 void printDiagnostic(const char* stage, const char* status) {
101     #if ENABLE_DIAGNOSTICS
102     Serial.print("[DIAG ");
103     Serial.print(stage);
104     Serial.print(" : ");
105     Serial.println(status);
106     #endif
107 }
108
109 void rainGaugeISR() {
110     unsigned long current_time = millis();
111     if (current_time - last_rain_time > DEBOUNCE_TIME) {
112         rain_pulses++;
113         last_rain_time = current_time;
114     }
115 }

```

La fonction `buttonISR` est appelée lors de chaque pression sur le bouton (avec un cooldown). La fonction `sendProgmem` envoie la commande LoRa depuis Flash vers le module :

```

120
121 void buttonISR() {
122     unsigned long currentTime = millis();
123
124     // Anti-rebond matériel
125     if (currentTime - lastInterruptTime >= BUTTON_COOLDOWN) {
126         buttonPressed = true;
127         lastInterruptTime = currentTime;
128
129         Serial.println("\n[BUTTON] INTERRUPTION DÉTECTÉE - ENVOI DES DONNÉES IMMINENT !");
130     } else {
131         Serial.println("[BUTTON] Veuillez ne pas appuyer trop rapidement sur le bouton, merci ! ");
132     }
133 }
134
135 // =====
136 // LORA & COMMUNICATION FUNCTIONS
137 // =====
138
139 /**
140  * Send AT command from PROGMEM
141  */
142 void sendProgmem(const char *cmd) {
143     char tmp[80];
144     strcpy_P(tmp, cmd);
145     loraSerial.print(tmp);
146     #if VERBOSE_MODE
147     Serial.print("[TX] ");
148     Serial.print(tmp);
149     #endif
150 }

```

Cette fonction permet de vérifier que la connexion LoRa s'est correctement effectuée :

```

/**
 * Send AT command and check for expected response
 */
static int at_send_check_response(const char *p_ack, int timeout_ms, const char *p_cmd) {
    int ch, index = 0;
    memset(recv_buf, 0, 200);
    sendProgmem(p_cmd);
    delay(50);

    if (!p_ack) return 0;

    unsigned long start = millis();
    do {
        while (loraSerial.available()) {
            recv_buf[index++] = loraSerial.read();
            if (index >= 199) index = 0;
            #if VERBOSE_MODE
            Serial.write(recv_buf[index-1]);
            #endif
            delay(2);
        }
        if (strstr(recv_buf, p_ack)) {
            printDiagnostic("Réponse LoRa", "OK - ACK reçu");
            return 1;
        }
    } while (millis() - start < timeout_ms);

    printDiagnostic("Réponse LoRa", "TIMEOUT - No ACK");
    return 0;
}

```


Toute cette partie du code permet d'encoder les valeurs extraites en hexadécimal :

```

186
187 /**
188  * Convert float to integer with scale
189  */
190 static long to_int(float value, int scale) {
191     return (long)round(value * scale);
192 }
193
194 /**
195  * Encode all sensor data to hexadecimal
196  */
197 void encodeDataToHex(char* hexOutput) {
198     printHeader("Processus d'encodage des données");
199
200     Serial.println("\n[ENCODE] Facteurs d'échelle:");
201     Serial.println(" - Échelle 100: Températures, Tensions, Énergies, Pressions");
202     Serial.println(" - Échelle 10: Humidités, Pressions");
203     Serial.println(" - Échelle 1: Puissance, Direction, Radiation");
204
205     int scale100 = 100;
206     int scale10 = 10;
207     int scale1 = 1;
208

```

```

208
209 // 14 uint16_t values: 9 from API + 1 from MS5637 + 2 from UV + 1 from Pyranometer + 1 from Pluviomètre
210 uint16_t hex_vals[14] = {
211     // API Data (9 values)
212     (uint16_t)to_int(api_vals[0], scale100), // [0] BattV_Avg (V)
213     (uint16_t)to_int(api_vals[1], scale10), // [1] AirT_C_Avg (°C)
214     (uint16_t)to_int(api_vals[2], scale10), // [2] RH (%)
215     (uint16_t)to_int(api_vals[3], scale1), // [3] SlrHoriz_W_Avg (W/m²)
216     (uint16_t)to_int(api_vals[4], scale100), // [4] SlrHoriz_MJ_Tot (MJ/m²)
217     (uint16_t)to_int(api_vals[5], scale1), // [5] SlrTilt_W_Avg (W/m²)
218     (uint16_t)to_int(api_vals[6], scale100), // [6] SlrTilt_MJ_Tot (MJ/m²)
219     (uint16_t)to_int(api_vals[7], scale100), // [7] WS_ms_S_WVT (m/s)
220     (uint16_t)to_int(api_vals[8], scale1), // [8] WindDir_D1_WVT (°)
221
222     // MS5637 Data (1 value)
223     (uint16_t)to_int(ms5637_pres, scale10), // [9] MS5637_Pres (hPa/mbar)
224
225     // UV Sensor Data (2 values)
226     (uint16_t)to_int(uv_voltage, scale1), // [10] UV_Voltage (mV)
227     (uint16_t)to_int(uv_index, scale1), // [11] UV_Index
228
229     // Pyranometer Data (1 value)
230     (uint16_t)to_int(radiation_Wm2, scale1), // [12] Pyrano_Radiation (W/m²)
231
232     // Données du pluviomètre
233     (uint16_t)to_int(rain_mm, scale10), // [13] Pluie_mm (mm)
234 };
235

```

```

236 // Affichage des valeurs avant conversion pour vérification
237 Serial.println("\n[ENCODE] Values before hex conversion:");
238 for(int i = 0; i < 14; i++) {
239     Serial.print("  ");
240     if(i < 10) Serial.print("0");
241     Serial.print(i);
242     Serial.print(" = ");
243     Serial.print(hex_vals[i]);
244     Serial.print(" -> 0x");
245     if(hex_vals[i] < 0x1000) Serial.print("0");
246     if(hex_vals[i] < 0x0100) Serial.print("0");
247     if(hex_vals[i] < 0x0010) Serial.print("0");
248     Serial.println(hex_vals[i], HEX);
249 }
250
251 // Conversion en hexadécimal (little-endian)
252 Serial.println("\n[ENCODE] Conversion en hexadécimal (little-endian):");
253 for(int i = 0; i < 14; i++) {
254     uint16_t v = hex_vals[i];
255     sprintf(&hexOutput[i*4], "%02X%02X", (v & 0xFF), (v >> 8));
256 }
257 hexOutput[56] = '\0'; // 14 * 4 = 56 characters (28 bytes)
258
259 Serial.print("\n[ENCODE] Payload finale : ");
260 Serial.println(hexOutput);
261 Serial.print("[ENCODE] Longueur de la Payload : ");
262 Serial.print(strlen(hexOutput));
263 Serial.println(" characters (28 bytes)");
264 }

```

Fonction permettant de récupérer la pression du MS5637 :

```
270 void readMS5637() {
271     Serial.println("\nLecture de données du capteur MS5637...");
272
273     if (!BaroSensor.isOK()) {
274         BaroSensor.begin(); // Réinitialiser
275         ms5637_pres = 0;
276         Serial.print(" Erreur: ");
277         Serial.println(BaroSensor.getError());
278         printDiagnostic("MS5637", "Lecture échouée - Tentative de réinitialisation");
279     } else {
280         // Lecture OK
281         ms5637_pres = BaroSensor.getPressure();
282
283         Serial.print(" Pressure: ");
284         Serial.print(ms5637_pres, 2);
285         Serial.println(" hPa/mbar");
286
287         printDiagnostic("MS5637", "Lecture réussie");
288     }
289 }
```

Fonction permettant d'obtenir l'indice UV depuis le capteur UV :

```
291 void readUVSensor() {
292     Serial.println("\nLecture de données du capteur UV...");
293
294     uv_voltage = UVIndex240370Sensor.readUvOriginalData();
295     uv_index = UVIndex240370Sensor.readUvIndexData();
296
297     Serial.print(" Voltage: ");
298     Serial.print(uv_voltage);
299     Serial.println(" mV");
300
301     Serial.print(" UV Index: ");
302     Serial.println(uv_index);
303
304     printDiagnostic("Capteur UV", "Lecture réussie");
305 }
```

Fonction permettant de récupérer des données depuis le pyranomètre :

```
307 void readPyranometer() {
308     Serial.println("\nLecture de données du capteur Pyranomètre...");
309
310     rawValue = analogRead(PYRANO_PIN);
311     voltage_mV = (rawValue / 1023.0) * 5000.0;
312     radiation_Wm2 = voltage_mV * PYRANO_CALIBRATION;
313
314     // Clamp to valid range
315     if (radiation_Wm2 < 0) radiation_Wm2 = 0;
316     if (radiation_Wm2 > 2000) radiation_Wm2 = 2000;
317
318     Serial.print(" Raw ADC: ");
319     Serial.println(rawValue);
320
321     Serial.print(" Voltage: ");
322     Serial.print(voltage_mV, 2);
323     Serial.println(" mV");
324
325     Serial.print(" Radiation Solaire: ");
326     Serial.print(radiation_Wm2, 1);
327     Serial.println(" W/m²");
328
329     printDiagnostic("Pyranomètre", "Lecture réussie");
330 }
331
```

Fonction permettant de récolter des données depuis le pluviomètre :

```
331
332 void readRainGauge() {
333     Serial.println("\nLecture données du pluviomètre...");
334
335     rain_mm = rain_pulses * RAIN_PER_PULSE;
336
337     Serial.print(" Pulsations: ");
338     Serial.println(rain_pulses);
339
340     Serial.print(" Pluie: ");
341     Serial.print(rain_mm, 2);
342     Serial.println(" mm");
343
344     // Réinitialiser pour le prochain cycle
345     rain_pulses = 0;
346
347     printDiagnostic("Pluviomètre", "Lecture réussie");
348 }
```

Fonction invoquant toutes les lectures de capteurs, appelée dans la boucle loop :

```
349
350 void readAllSensors() {
351     printHeader("Collecte des données de tous les capteurs");
352     readMS5637();
353     readUVSensor();
354     readPyranometer();
355     readRainGauge();
356     Serial.println();
357 }
```

La fonction « DisplayAPIData » permet d'afficher les données reçues depuis l'API :

```
363 void displayAPIData() {
364     Serial.println("\nDonnées météo reçues de l'API:");
365     Serial.print(" [0] BattV_Avg: ");
366     Serial.print(api_vals[0], 2);
367     Serial.println(" V");
368
369     Serial.print(" [1] AirT_C_Avg: ");
370     Serial.print(api_vals[1], 2);
371     Serial.println(" °C");
372
373     Serial.print(" [2] RH: ");
374     Serial.print(api_vals[2], 2);
375     Serial.println(" %");
376
377     Serial.print(" [3] SlrHoriz_W_Avg: ");
378     Serial.print(api_vals[3], 2);
379     Serial.println(" W/m²");
380
381     Serial.print(" [4] SlrHoriz_MJ_Tot: ");
382     Serial.print(api_vals[4], 2);
383     Serial.println(" MJ/m²");
384
385     Serial.print(" [5] SlrTilt_W_Avg: ");
386     Serial.print(api_vals[5], 2);
387     Serial.println(" W/m²");
388
389     Serial.print(" [6] SlrTilt_MJ_Tot: ");
390     Serial.print(api_vals[6], 2);
391     Serial.println(" MJ/m²");
392
393     Serial.print(" [7] WS_ms_S_WVT: ");
394     Serial.print(api_vals[7], 2);
395     Serial.println(" m/s");
}
```

La fonction fetchWeatherDataAPI récolte des données depuis l'API :

```
396
397     Serial.print(" [8] WindDir_D1_WVT: ");
398     Serial.print(api_vals[8], 2);
399     Serial.println(" °");
400 }
401
402 void fetchWeatherDataAPI() {
403     printHeader("Collecte de données depuis l'API Météo");
404
405     Serial.println("\n[API] Connexion au serveur...");
406     Serial.print(" Host: ");
407     Serial.print(TCP_HOST);
408     Serial.print(":");
409     Serial.println(TCP_PORT);
410
411     if (!ethClient.connected()) {
412         Serial.println(" Non connecté, initialisation d'une nouvelle connexion...");
413         if (!ethClient.connect(TCP_HOST, TCP_PORT)) {
414             printDiagnostic("API", "Connection échouée");
415             delay(1000);
416             return;
417         }
418         printDiagnostic("API", "Connection réussie");
419         delay(500);
420     }
```

```
421
422     if (ethClient.connected()) {
423         Serial.println("\nEnvoi de la requête HTTP...");
424         ethClient.println("GET /command=DataQuery&uri=MeteoPau&format=json&mode=backfill&p1=600 HTTP/1.1");
425         ethClient.print("Host: ");
426         ethClient.println(TCP_HOST);
427         ethClient.println("Connection: close\r\n");
428         printDiagnostic("API", "Requête envoyée");
429     }
430
431     delay(1000);
432
433     rxIndex = 0;
434     memset(rxBuffer, 0, 1024);
435
436     Serial.println("\n[API] Lecture de la réponse HTTP...");
437     unsigned long readStart = millis();
438     while (ethClient.available()) {
439         char c = ethClient.read();
440         if (rxIndex < 1023) {
441             rxBuffer[rxIndex++] = c;
442         }
443     }
444     unsigned long readTime = millis() - readStart;
445     rxBuffer[rxIndex] = '\0';
446
447     Serial.print(" Taille de la réponse : ");
448     Serial.print(rxIndex);
449     Serial.println(" bytes");
450     Serial.print(" Temps de lecture: ");
451     Serial.print(readTime);
452     Serial.println(" ms");
```

```
453
454     #if VERBOSE_MODE
455     Serial.println("\n[API] Réponse raw (LAST 150 chars):");
456     int start = (rxIndex > 150) ? rxIndex - 150 : 0;
457     for(int i = start; i < rxIndex; i++) {
458         Serial.write(rxBuffer[i]);
459     }
460     Serial.println("\n");
461     #endif
462
463     debugAPIResponse();
464
465     Serial.println("[API] Parsing des données JSON...");
466     parseWeatherData(rxBuffer);
467
468     displayAPIData();
469
470     ethClient.stop();
471     printDiagnostic("API", "Connection fermée");
472 }
473
```

```

474 void parseWeatherData(const char* json) {
475     printDiagnostic("Parser", "Recherche de la structure JSON...");
476
477     // Cherche soit "data" (ancien format) soit "rows" (nouveau format)
478     const char* valsSource = strstr(json, "\"data\":");
479     if (!valsSource) {
480         valsSource = strstr(json, "\"rows\":");
481         if (!valsSource) {
482             printDiagnostic("Parser", "ERREUR - Aucun tableau 'data' ou 'rows' trouvé");
483             return;
484         }
485         printDiagnostic("Parser", "'rows' array found (NEW FORMAT) ");
486     } else {
487         printDiagnostic("Parser", "'data' array found (OLD FORMAT) ");
488     }
489
490     // Cherche le premier [ (tableau de données)
491     const char* arrayStart = strstr(valsSource, "[");
492     if (!arrayStart) {
493         printDiagnostic("Parser", "ERREUR - Pas de tableau trouvé");
494         return;
495     }
496
497     // Cherche le second [ (les valeurs réelles)
498     arrayStart = strstr(arrayStart + 1, "[");
499     if (!arrayStart) {
500         printDiagnostic("Parser", "ERREUR - Pas de tableau de valeurs trouvé");
501         return;
502     }

```

```

503
504     arrayStart += 1; // Skip the [
505
506     int valIndex = 0;
507     char numStr[20];
508     int numIndex = 0;
509     int skipCount = 0; // Pour skipper les 3 premiers éléments (id, time, no)
510
511     Serial.println("\n  Extraction des valeurs:");
512
513     // Initialiser toutes les valeurs à 0
514     for(int i = 0; i < 9; i++) {
515         api_vals[i] = 0.0;
516     }
517
518     // Parser la chaîne
519     for (int i = 0; arrayStart[i]; i++) {
520         char c = arrayStart[i];
521
522         if (c == ']') {
523             // Traiter la dernière valeur si elle existe
524             if (numIndex > 0) {
525                 numStr[numIndex] = '\0';
526
527                 // Skipper les 3 premiers éléments
528                 if (skipCount >= 3 && valIndex < 9) {
529                     api_vals[valIndex] = atof(numStr);
530
531                     Serial.print("    ");
532                     if(valIndex < 10) Serial.print("0");
533                     Serial.print(valIndex);
534                     Serial.print("] = ");
535                     Serial.println(numStr);

```

```

536                 valIndex++;
537             }
538             skipCount++;
539         }
540         break;
541     }
542
543     // Parser les nombres
544     if ((c >= '0' && c <= '9') || c == '.' || c == '-') {
545         if (numIndex < 19) numStr[numIndex++] = c;
546     }
547
548     // Séparateur (virgule)
549     else if (c == ',' && numIndex > 0) {
550         numStr[numIndex] = '\0';
551
552         // Skipper les 3 premiers éléments (id, time, no)
553         if (skipCount >= 3 && valIndex < 9) {
554             api_vals[valIndex] = atof(numStr);
555
556             Serial.print("    ");
557             if(valIndex < 10) Serial.print("0");
558             Serial.print(valIndex);
559             Serial.print("] = ");
560             Serial.println(numStr);
561

```

```

560         Serial.println(numStr);
561     }
562     valIndex++;
563 }
564 skipCount++;
565 numIndex = 0;
566 }
567 }
568
569 Serial.print("\n[Parser] Extraction complete - ");
570 Serial.print(valIndex);
571 Serial.println(" valeurs extraites |");
572 }

```

Fonction qui permet d'envoyer urgemment une alerte en LoRa lorsque la tension moyenne de la batterie dépasse un certain seuil :

```

577
578 void sendBatteryAlert() {
579     printHeader("==== ALERTE BATTERIE FAIBLE - ENVOI D'URGENCE =====");
580
581     Serial.println("\n[ALERTE] BATTERIE FAIBLE!");
582     Serial.print("[ALERTE] Tension moyenne : ");
583     Serial.print(api_vals[0], 2);
584     Serial.println(" V");
585
586     // Créer un payload d'alerte SIMPLIFIÉ: seulement batterie + code alerte
587     // 2 uint16_t: batterie (scale 100) + code alerte
588     char alertPayload[17]; // 8 bytes = 16 caractères hex + 1 null terminator
589
590     uint16_t battery_val = (uint16_t)to_int(api_vals[0], 100); // Batterie avec scale 100
591     uint16_t alert_code = ALERT_BATTERY_LOW; // 0xFF01
592
593     // Encoder en little-endian
594     sprintf(alertPayload, "%02X%02X%02X%02X",
595         (battery_val & 0xFF), (battery_val >> 8),
596         (alert_code & 0xFF), (alert_code >> 8)
597     );
598
599     Serial.print("[ALERTE] Payload: ");
600     Serial.println(alertPayload);
601     Serial.print("[ALERTE] Longueur: ");
602     Serial.print(strlen(alertPayload));
603     Serial.println(" characters (8 bytes)");
604 }

```

```

604
605 // Transmettre l'alerte
606 printHeader("Transmission de l'alerte batterie via LoRa");
607
608 Serial.println("\nPréparation de la transmission LoRa...");
609 Serial.print(" Payload: ");
610 Serial.println(alertPayload);
611
612 char cmd[96];
613 sprintf(cmd, "AT+CMSSGHEX=\"%s\"\r\n", alertPayload);
614
615 Serial.println("\n[LoRa] Envoi de la commande AT...");
616 loraSerial.print(cmd);
617
618 Serial.println("\n[LoRa] Attente de la confirmation de transmission...");
619 if (at_send_check_response("Done", 5000, "")) {
620     printDiagnostic("LoRa TX ALERTE", "Transmission réussie");
621 } else {
622     printDiagnostic("LoRa TX ALERTE", "Transmission échouée");
623 }
624
625 Serial.println();
626 }
627

```

Fonction de transmission des données en LoRa :

```
628
629 // =====
630 // LORA TRANSMISSION
631 // =====
632
633 void transmitDataLoRa(const char* hexPayload) {
634     printHeader("Transmission des données via LoRa");
635
636     Serial.println("\nPréparation de la transmission LoRa...");
637     Serial.print(" Payload: ");
638     Serial.println(hexPayload);
639     Serial.print(" Longueur de la Payload: ");
640     Serial.print(strlen(hexPayload));
641     Serial.println(" characters");
642
643     char cmd[96];
644     sprintf(cmd, "AT+MSGHEX=\"%s\"\\r\\n", hexPayload);
645
646     Serial.println("\n[LoRa] Envoi de la commande AT...");
647     Serial.print(" Commande: ");
648     Serial.println(cmd);
649
650     loraSerial.print(cmd);
651
652     Serial.println("\n[LoRa] Attente de la confirmation de transmission...");
653     if (at_send_check_response("Done", 5000, "")) {
654         printDiagnostic("LoRa TX", "Transmission réussie");
655     } else {
656         printDiagnostic("LoRa TX", "Transmission échoué ou timeout");
657     }
658
659     Serial.println();
660 }
661
```

Fonction de scan des périphériques I2C (Utilisé pour le debug) :

```
662 // =====
663 // I2C SCANNING
664 // =====
665
666 void testI2C() {
667     Serial.println("Scan des périphériques I2C...");
668
669     Wire.begin();
670     int devices = 0;
671     for(byte i = 8; i < 120; i++) {
672         Wire.beginTransmission(i);
673         if(Wire.endTransmission() == 0) {
674             Serial.print(" Périphérique I2C trouvé à 0x");
675             Serial.println(i, HEX);
676             devices++;
677         }
678     }
679
680     Serial.print("Trouvés ");
681     Serial.print(devices);
682     Serial.println(" Périphériques I2C\n");
683 }
684
```

Fonction setup permettant initialisation de tous les composants :

```
688
689 void setup() {
690     Serial.begin(9600);
691     delay(2000);
692
693     printHeader("==== Démarrage du Diagnostic ====");
694
695     printDiagnostic("Système", "Boot initialisé");
696
697     // ===== BUTTON INIT =====
698     Serial.println("\n[BUTTON] Initialisation du bouton d'urgence sur la broche 7...");
699     pinMode(BUTTON_PIN, INPUT_PULLUP);
700     attachInterrupt(digitalPinToInterrupt(BUTTON_PIN), buttonISR, FALLING);
701     printDiagnostic("BUTTON", "Initialisé avec interruption matérielle");
702
703     // ===== SERIAL1 INIT =====
704     Serial.println("\n[UV Sensor] Initialisation sur Serial1 (D0-D1)...");
705     Serial1.begin(9600);
706     delay(100);
707     printDiagnostic("Serial1 UV", "OK");
708
709     // ===== SOFTWARESERIAL INIT =====
710     Serial.println("\n[SOFTWARESERIAL] Initialisation sur les broches 2-3...");
711     loraSerial.begin(9600);
712     delay(100);
713     printDiagnostic("SoftwareSerial", "OK");
714 }
```

```
715 // ===== ETHERNET SHIELD INIT =====
716 Serial.println("\n[ETHERNET] Initialisation du Shield Ethernet...");
717 Serial.println(" Réinitialisation de la broche CS (broche 10)...");
718 pinMode(10, OUTPUT);
719 digitalWrite(10, LOW);
720 delay(100);
721 digitalWrite(10, HIGH);
722 delay(500);
723 printDiagnostic("Ethernet", "Réinitialisation de la broche CS terminée");
724
725 Serial.println("\n[ETHERNET] Configuration de l'IP Statique...");
726
727 // ===== CONFIGURATION IP STATIQUE =====
728 IPAddress ip(150, 100, 100, 67); // IP Arduino Pour API : 150.100.100.67
729 IPAddress gateway(150, 100, 100, 20); // Passerelle (votre routeur) Pour API : 150.100.100.20
730 IPAddress subnet(255, 255, 255, 0); // Masque de sous-réseau
731 IPAddress dns(150, 100, 100, 20); // DNS (votre routeur) Pour API : 150.100.100.20
732 Ethernet.begin(mac, ip, dns, gateway, subnet);
733
734 Serial.print(" Adresse IP : ");
735 Serial.println(Ethernet.localIP());
736
737
738 Serial.print(" Adresse IP (statique): ");
739 Serial.println(Ethernet.localIP());
740 Serial.print(" Passerelle: ");
741 Serial.println(gateway);
742 Serial.print(" Masque: ");
743 Serial.println(subnet);
744 Serial.print(" DNS: ");
745 Serial.println(dns);
746
747 printDiagnostic("Ethernet", "Adresse IP statique configurée");
```



```

748 // ===== I2C INIT =====
749 Serial.println("\n[I2C] Démarrage de Wire (broches SDA/SCL)...");
750 Wire.begin();
751 delay(100);
752 printDiagnostic("I2C", "Initialisé");
753
754
755 testI2C();
756
757 // ===== MS5637 INIT =====
758 Serial.println("[MS5637] Initialisation du capteur...");
759 int ms5637_attempts = 0;
760 while (!BaroSensor.isOK() && ms5637_attempts < 20) {
761     Serial.print(".");
762     BaroSensor.begin();
763     ms5637_attempts++;
764     delay(1000);
765 }
766 if (ms5637_attempts < 20) {
767     printDiagnostic("MS5637", "Initialisé avec succès");
768 } else {
769     printDiagnostic("MS5637", "ÉCHEC DE L'INITIALISATION après 20 tentatives");
770 }
771
772 // ===== UV SENSOR INIT =====
773 Serial.println("[Capteur UV] Initialisation du capteur...");
774 int uv_attempts = 0;
775 while (!UVIndex240370Sensor.begin() && uv_attempts < 20) {
776     Serial.print(".");
777     uv_attempts++;
778     delay(1000);
779 }

```

```

779 }
780 if (uv_attempts < 20) {
781     printDiagnostic("Capteur UV", "Initialisé avec succès");
782 } else {
783     printDiagnostic("Capteur UV", "ÉCHEC DE L'INITIALISATION après 20 tentatives");
784 }
785
786 // ===== RAIN GAUGE INIT =====
787 Serial.println("\n[Pluviomètre] Initialisation du Pluviomètre...");
788 pinMode(RAIN_GAUGE_PIN, INPUT_PULLUP);
789 attachInterrupt(digitalPinToInterrupt(RAIN_GAUGE_PIN), rainGaugeISR, FALLING);
790 printDiagnostic("Pluviomètre", "Pluviomètre initialisé sur le pin 8");
791
792 // ===== PYRANOMETER TEST =====
793 Serial.println("\n[Pyranomètre] Test de l'input analogue...");
794 int pyrano_raw = analogRead(PYRANO_PIN);
795 float pyrano_voltage = (pyrano_raw / 1023.0) * 5000.0;
796 Serial.print(" Raw ADC: ");
797 Serial.println(pyrano_raw);
798 Serial.print(" Voltage: ");
799 Serial.println(pyrano_voltage, 2);
800 Serial.println(" mV");
801 printDiagnostic("Pyranomètre", "Analog input OK");
802

```

```

803 // ===== WIO-E5 LORA MODULE INIT =====
804 Serial.println("\n[WIO-E5] Test de la connexion LoRa au module..");
805 if (at_send_check_response("+AT: OK", 200, STR_AT)) {
806     lora_module_exists = true;
807     printDiagnostic("Wio-E5", "Module trouvé et répondant");
808
809     Serial.println("\n[WIO-E5] Envoi des commandes de configuration...");
810
811     Serial.println(" [1/6] ID du module...");
812     at_send_check_response("+ID:", 1000, STR_ID);
813
814     Serial.println(" [2/6] Mode LWOTAA...");
815     at_send_check_response("+MODE: LWOTAA", 1000, STR_MODE);
816
817     Serial.println(" [3/6] Région configurée à EU868...");
818     at_send_check_response("+DR:", 1000, STR_DR);
819
820     Serial.println(" [4/6] Configuration de l'APP key...");
821     at_send_check_response("+KEY: APPKEY", 1000, STR_KEY);
822
823     Serial.println(" [5/6] Configuration Classe A...");
824     at_send_check_response("+CLASS:", 1000, STR_CLASS);
825
826     Serial.println(" [6/6] Configuration du port 8...");
827     at_send_check_response("+PORT:", 1000, STR_PORT);
828
829     lora_network_joined = true;
830     printDiagnostic("Wio-E5", "Configuration terminée");
831 } else {
832     lora_module_exists = false;
833     printDiagnostic("Wio-E5", "Module non trouvé ou ne répond pas");
834 }
835 printHeader("===== Setup Complete =====");

```

```

840 void debugAPIResponse() {
841     printHeader("DEBUG: API RESPONSE ANALYSIS");
842
843     Serial.println("\n[DEBUG] Response Statistics:");
844     Serial.print(" Total bytes: ");
845     Serial.println(rxIndex);
846
847     Serial.println("\n[DEBUG] COMPLETE RAW RESPONSE:");
848     Serial.println("---START---");
849     for(int i = 0; i < rxIndex; i++) {
850         Serial.write(rxBuffer[i]);
851     }
852     Serial.println("\n---END---");
853
854     Serial.println("\n[DEBUG] JSON PART ONLY:");
855     const char* jsonStart = strstr(rxBuffer, "{");
856     if (jsonStart) {
857         int indent = 0;
858         bool inString = false;
859         for(int i = 0; jsonStart[i]; i++) {
860             char c = jsonStart[i];
861
862             if (c == '"') {
863                 inString = !inString;
864             }
865
866             if (!inString) {
867                 if (c == '{' || c == '[') {
868                     Serial.write(c);
869                     Serial.write('\n');
870                     indent += 2;
871                     for(int j = 0; j < indent; j++) Serial.write(' ');

```

```

872     }
873     else if (c == '}' || c == ']') {
874         Serial.write('\n');
875         indent = (indent > 2) ? indent - 2 : 0;
876         for(int j = 0; j < indent; j++) Serial.write(' ');
877         Serial.write(c);
878     }
879     else if (c == ',') {
880         Serial.write(',');
881         Serial.write('\n');
882         for(int j = 0; j < indent; j++) Serial.write(' ');
883     }
884     else if (c == ':') {
885         Serial.write(": ");
886     }
887     else if (c != ' ' && c != '\n' && c != '\r' && c != '\t') {
888         Serial.write(c);
889     }
890     } else {
891         Serial.write(c);
892     }
893 }
894 Serial.println("\n");
895 } else {
896     Serial.println("ERROR: No JSON found!");
897 }

```

```

897     }
898
899     Serial.println("\n[DEBUG] JSON Structure:");
900     int braces = 0, brackets = 0;
901     for(int i = 0; i < rxIndex; i++) {
902         if (rxBuffer[i] == '{') braces++;
903         if (rxBuffer[i] == '}') braces--;
904         if (rxBuffer[i] == '[') brackets++;
905         if (rxBuffer[i] == ']') brackets--;
906     }
907     Serial.print("  Brace balance: ");
908     Serial.println(braces);
909     Serial.print("  Bracket balance: ");
910     Serial.println(brackets);
911
912     if (braces == 0 && brackets == 0) {
913         Serial.println("    structure Json valide");
914     } else {
915         Serial.println("    structure Json invalide");
916     }
917 }
918

```

```

923 void loop() {
924     if(!lora_module_exists) {
925         delay(2000);
926         return;
927     }
928
929     // ===== BUTTON CHECK (POLLING) =====
930     static unsigned long lastButtonTime = 0;
931     static bool buttonWasPressed = false;
932
933     if (digitalRead(BUTTON_PIN) == HIGH) { // ← HIGH quand appuyé (car INPUT_PULLUP)
934         // Bouton appuyé
935         if (!buttonWasPressed && (millis() - lastButtonTime >= BUTTON_COOLDOWN)) {
936             buttonPressed = true;
937             lastButtonTime = millis();
938             buttonWasPressed = true;
939             Serial.println("\n[BUTTON] INTERRUPTION DÉTECTÉE - ENVOI DES DONNÉES IMMINENT !");
940         }
941     } else {
942         // Bouton relâché
943         buttonWasPressed = false;
944     }
945
946     // ===== LORA NETWORK JOIN =====
947     if (lora_network_joined) {
948         Serial.println("\n[LoRa] Tentative de connexion au reseau...");
949
950         char tmp[80];
951         strcpy_P(tmp, STR_JOIN);
952         loraSerial.print(tmp);
953
954         memset(recv_buf, 0, 200);
955         delay(100);

```

```

957     int index = 0;
958     unsigned long start = millis();
959     bool network_joined = false;
960
961     while (millis() - start < 12000) {
962         while (loraSerial.available()) {
963             recv_buf[index++] = loraSerial.read();
964             if(index >= 199) index = 0;
965             #if VERBOSE_MODE
966                 Serial.write(recv_buf[index-1]);
967             #endif
968             delay(2);
969         }
970
971         if (strstr(recv_buf, "+JOIN: Network joined") ||
972             strstr(recv_buf, "+JOIN: Joined already")) {
973             network_joined = true;
974             break;
975         }
976
977         if (strstr(recv_buf, "+JOIN: Join failed")) {
978             printDiagnostic("LoRa", "Join failed - Nouvelle tentative");
979             lora_network_joined = true;
980             delay(10000);
981             return;
982         }
983     }
984

```

```

985     if (network_joined) {
986         lora_network_joined = false;
987         printDiagnostic("LoRa", "Connexion au réseau réussie ");
988         Serial.println("[LoRa] Passage à la boucle de collecte de données\n");
989     } else {
990         printDiagnostic("LoRa", "Timeout - Nouvelle tentative dans 10s");
991         delay(10000);
992         return;
993     }
994 }
995
996 // ===== COLLECTE DE DONNÉES (NORMALE OU FORCÉE PAR INTERRUPTION) =====
997 bool shouldCollect = false;
998
999 // Cas 1: Bouton pressé (interruption)
1000 if (buttonPressed) {
1001     shouldCollect = true;
1002     buttonPressed = false; // Réinitialiser le flag
1003     printHeader("===== ENVOI D'URGENCE (INTERRUPTION) =====");
1004 }
1005
1006 // Cas 2: Intervalle de 3 minutes écoulé (envoi régulier)
1007 else if (millis() - lastSendTime >= SEND_INTERVAL) {
1008     shouldCollect = true;
1009     printHeader("===== Cycle régulier (3 minutes) =====");
1010 }
1011
1012 // Exécuter l'envoi si l'une des conditions est vraie
1013 if (shouldCollect) {
1014     lastSendTime = millis();
1015
1016     Serial.print("[CYCLE] Timestamp: ");
1017     Serial.println(millis());
1018     Serial.println("[CYCLE] Lecture de toutes les sources de donnees...\n");

```

```

1020 // Fetch API data
1021 fetchWeatherDataAPI();
1022
1023 // Read all sensors
1024 readAllSensors();
1025
1026 if (api_vals[0] < 12.0) {
1027     BattV_Avg_is_low = true;
1028
1029     if (millis() - lastBatteryAlertTime >= 300000) { // 5 minutes
1030         sendBatteryAlert();
1031         lastBatteryAlertTime = millis();
1032         delay(2000);
1033         printHeader("===== Alerte batterie envoyée =====");
1034     } else {
1035         printDiagnostic("BATTERIE", "Alerte déjà envoyée récemment");
1036     }
1037 } else {
1038     BattV_Avg_is_low = false;
1039 }
1040
1041 // Encode data to hex
1042 char hexPayload[57]; // 56 + 1 for null terminator
1043 encodeDataToHex(hexPayload);
1044
1045 // Transmit via LoRa
1046 transmitDataLoRa(hexPayload);
1047
1048 printHeader("===== Cycle Terminé, le prochain cycle commence et se terminera dans 15 minutes =====");
1049 }
1050
1051 delay(100);

```

4.Fausse API :

Dans ce repo, vous trouverez une fausse API. Cette API vise à imiter l'API du Datalogger afin de pouvoir effectuer une série de tests. (Cette fausse API a été réalisée sans pouvoir voir ce à quoi ressemble la véritable API, elle n'est pas totalement fidèle à la réalité).

Cette API est un programme python, afin de la lancer, téléchargez le fichier puis placez vous dans le même dossier et effectuez :

« source venv/bin/activate » puis **« python ServeurAPI_3.py »** ou **« python3 ServeurAPI_3.py »**.

Une fois lancée, l'API va attendre de recevoir une connexion, puis envoyer des fausses données météo générées aléatoirement.

Il y a deux versions de la fausse API : Une version faite pour tourner sur un Raspberry PI et une autre pour un PC personnel. Les différences se situent à la fin du programme dans la partie « server address » et sur l'URL de l'adresse. Ces lignes sont modifiables selon les besoins.

Lancez les programmes Arduino en même temps que l'API pour qu'ils puissent récupérer les données. Une fois les deux programmes en fonctionnement, l'Arduino va envoyer un code toutes les X minutes qui va être décodé par le Codec sur Chirpstack.

4.Codec Chirpstack :

Le programme Codec.js est à placer dans Chirpstack dans la partie *Device Profiles/Payload Codec* et dans son Device choix (dans notre cas l'Arduino). Ce programme va gérer les réceptions LoRa et décoder la payload en hexadécimal afin d'afficher les valeurs reçues depuis les capteurs dans la partie Event de l'Application utilisant ce capteur.

2. Les messages comprenant **DevAddr** (Adresse hexadécimale du Wio) il s'agit d'un identifiant unique (comme une IP LoRa) transmis à chaque join, il permet de s'assurer qu'une connexion entre chirpstack et l'arduino a eu lieu.

2026-06-19 13:44:45  DevAddr: 09726388

3. Les messages comprenant **Code : OTAA** (Over The Air Activation = Demande au serveur une autorisation à chaque JOIN)

2026-06-19 13:44:45  Code: OTAA Level: ERROR

V. Conclusion :

Ce projet permet de réceptionner des données via une API et un ensemble de capteurs, puis d'encoder ces données sous forme d'une chaîne de caractères en hexadécimal avant de les envoyer en LoRa sur Chirpstack afin de les exploiter.